

# Week 1 Lab: Baseline Firmware, C Programming Review, and Measurement Habits

EE0827 Embedded System Applications

## Overview

Build a known-good baseline project on the STM32F401RE Nucleo. You will implement UART logging, a timer tick with one ISR, and a GPIO trace pin, then capture timing evidence using an oscilloscope. This baseline becomes the reference for all later labs.

This lab also reinforces the C programming concepts covered in theory: pointers, structs, volatile, and common pitfalls.

**Time allocation:** 90 minutes

- Setup and CubeMX configuration: 20 min
- Firmware implementation: 30 min
- Measurements and evidence capture: 25 min
- Documentation and cleanup: 15 min

## Learning Objectives

By the end of this lab, you will be able to:

1. Configure core peripherals (GPIO, UART, timer interrupt) using STM32CubeMX and write firmware in STM32CubeIDE
2. Implement a timer ISR with a trace pin for timing visibility
3. Create observable firmware with UART logging
4. Capture timing evidence with an oscilloscope
5. Demonstrate understanding of C pointers by explaining code behavior
6. Produce a reproducible demo with documented wiring and setup

## Safety Gate

### Safety Checklist (Complete Before Powering On)

- ☐ Power source: USB only OR bench supply only (NEVER both simultaneously)
- ☐ If using bench supply: current limit set to 200 mA before power-on
- ☐ USB cable connected to ST-LINK port, not CN5 user USB
- ☐ No external signals or wires connected to header pins
- ☐ Oscilloscope probe ground clip connected to GND header pin only

**Expected current draw:** 25-50 mA idle (USB powered)

### Common Ways to Destroy the Board

1. **Dual power:** Powering from USB and bench supply at the same time can damage both the board and your computer's USB port.
2. **5V to GPIO:** Connecting 5V signals to 3.3V GPIO pins can destroy the pin or the entire MCU.
3. **Short circuits:** Metal tools or probe tips touching adjacent pins can create shorts.
4. **Reversed polarity:** Connecting bench supply with wrong polarity damages voltage regulators.

**This procedure avoids these by using a single power source and no external signals.**

## Required Parts and Tools

### Hardware

- STM32F401RE Nucleo board
- USB Mini-B cable (for ST-LINK)
- Oscilloscope with probes (required)
- Multimeter (for basic verification)
- Jumper wires (for oscilloscope connection)

### Software

- STM32CubeMX (for peripheral configuration and code generation)
- STM32CubeIDE 2.0 (for writing and debugging firmware)

- Terminal program (PuTTY, Tera Term, or built-in serial monitor)

## ! PRE-LAB SECTION

**Complete the following BEFORE arriving at the lab session.** You will not have time to complete these during the lab period.

### Pre-Lab Checklist

Complete before arriving at the lab:

- ☐ STM32CubeMX and STM32CubeIDE 2.0 installed and verified
- ☐ Read STM32F401RE datasheet: GPIO section (absolute maximum ratings)
- ☐ Read Reference Manual: TIM2 section (pages 1-3 overview)
- ☐ Read Reference Manual: USART2 section (pages 1-2 overview)
- ☐ Complete C Programming Self-Assessment (see theory slides)

### C Programming Self-Assessment

Answer these questions before lab. You should be able to explain any of them.

1. What does `ptr++` do if `ptr` is declared as `uint32_t *ptr`?
2. When should you use the `volatile` keyword in embedded systems?
3. In the callback, what is `htim->Instance`? How does the `->` operator work?

## IN-LAB SECTION

The following sections are completed during the lab session.

### Part 1 Project Setup

*20 minutes*

#### Step 1.1: Create New Project in CubeMX

1. Open **STM32CubeMX**
2. Click **New Project**
3. Select **Board Selector** tab
4. Search for “NUCLEO-F401RE” and select it
5. Click **Start Project**
6. When prompted “Initialize all peripherals with their default Mode?”, click **Yes**

#### Step 1.2: Configure USART2

The Nucleo board uses USART2 (PA2/PA3) for the ST-LINK virtual COM port.

1. Verify USART2 is enabled in the pinout view
2. Click on **USART2** in the left panel
3. Verify settings:
  - Mode: Asynchronous
  - Baud Rate: 115200
  - Word Length: 8 Bits
  - Parity: None
  - Stop Bits: 1

**Note:** PA2 = TX, PA3 = RX (already configured by default)

#### Step 1.3: Configure Trace Pin

Choose a GPIO pin for the trace output. We recommend **PA5** (connected to user LED LD2).

1. In the CubeMX pinout view, click on pin **PA5**
2. Select **GPIO\_Output**
3. Right-click PA5 → Enter User Label: **TRACE**
4. In GPIO configuration (left panel → GPIO):

- Mode: Output Push Pull
- Pull: No pull-up and no pull-down
- Speed: High

### Step 1.4: Configure TIM2

Configure Timer 2 to generate an interrupt at 1 kHz (every 1 ms).

1. Click **Timers** → **TIM2** in the left panel
2. Set Clock Source: **Internal Clock**
3. In Parameter Settings:
  - Prescaler: 8399 ( $84 \text{ MHz} / 8400 = 10 \text{ kHz}$ )
  - Counter Period: 9 ( $10 \text{ kHz} / 10 = 1 \text{ kHz}$ )
  - Counter Mode: Up
4. In NVIC Settings tab:
  - Enable **TIM2 global interrupt**

#### Calculation:

Timer Clock = 84 MHz (APB1 timer clock)  
 Prescaler = 8400 (register value = 8399)  
 Counter Period = 10 (register value = 9)  
 Update Frequency =  $84 \text{ MHz} / 8400 / 10 = 1000 \text{ Hz} = 1 \text{ kHz}$

### Step 1.5: Generate Code and Open in CubeIDE

1. In CubeMX: **Project Manager** tab → set Project Name to `week01_baseline`
2. Set Toolchain/IDE to **STM32CubeIDE**
3. Click **Generate Code**
4. Open **STM32CubeIDE 2.0** → **File** → **Import** → **Existing Projects into Workspace**
5. Browse to the generated project folder and import it

### Step 2.1: Implement Timer ISR with Trace Pin

Open `Core/Src/stm32f4xx_it.c` and find the `TIM2_IRQHandler` function.

Add trace pin toggle at entry and exit:

```
void TIM2_IRQHandler(void)
{
    /* USER CODE BEGIN TIM2_IRQn 0 */
    // Set trace pin HIGH at ISR entry
    HAL_GPIO_WritePin(TRACE_GPIO_Port, TRACE_Pin, GPIO_PIN_SET);
    /* USER CODE END TIM2_IRQn 0 */

    HAL_TIM_IRQHandler(&htim2);

    /* USER CODE BEGIN TIM2_IRQn 1 */
    // Set trace pin LOW at ISR exit
    HAL_GPIO_WritePin(TRACE_GPIO_Port, TRACE_Pin, GPIO_PIN_RESET);
    /* USER CODE END TIM2_IRQn 1 */
}
```

### Step 2.2: Declare Global Tick Counter

Open `Core/Src/main.c` and add a global variable after the includes:

```
/* USER CODE BEGIN PV */
volatile uint32_t tick_count = 0;
/* USER CODE END PV */
```

**Why volatile?** The variable is modified in the ISR and read in main. Without `volatile`, the compiler might optimize away the read in main, causing incorrect behavior.

### Step 2.3: Increment Counter in ISR

We need a callback function. Add to `main.c`:

```

/* USER CODE BEGIN 0 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        tick_count++;
    }
}
/* USER CODE END 0 */

```

## Step 2.4: Implement UART Logging Function

Add a simple logging function to `main.c`:

```

/* USER CODE BEGIN 0 */
#include <stdio.h>
#include <string.h>

// Redirect printf to UART (one character at a time)
int __io_putchar(int ch)
{
    HAL_UART_Transmit(&huart2, (uint8_t*)&ch, 1, HAL_MAX_DELAY);
    return ch;
}

void log_info(const char *message)
{
    printf("[INFO] %s\r\n", message);
}

void log_value(const char *name, uint32_t value)
{
    printf("[INFO] %s = %lu\r\n", name, value);
}
/* USER CODE END 0 */

```

## Step 2.5: Start Timer and Log Boot Message

In `main()`, after peripheral initialization, add:



```

/* USER CODE BEGIN 2 */
// Start TIM2 with interrupt
HAL_TIM_Base_Start_IT(&htim2);

// Boot banner
log_info("=====");
log_info("EE0827 Week 1 Baseline Firmware");
log_info("UART: 115200 8N1");
log_info("Timer: TIM2 @ 1 kHz");
log_info("=====");
log_info("System initialized.");
/* USER CODE END 2 */

```

## Step 2.6: Add Periodic Status Output

In the main loop:

```

/* USER CODE BEGIN WHILE */
while (1)
{
    // Print tick count every second (1000 ticks)
    static uint32_t last_print = 0;
    if (tick_count - last_print >= 1000)
    {
        last_print = tick_count;
        log_value("tick_count", tick_count);
    }
}
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */

```

## Step 2.7: Build and Verify

1. **Project** → **Build Project** (or Ctrl+B)
2. Verify: **0 errors, 0 warnings**
3. If warnings appear, fix them before proceeding

### Checkpoint 1: Basic Functionality

Before proceeding to measurements, verify:

- ☐ Project builds with no errors and no warnings
- ☐ UART output visible in terminal (115200 baud)
- ☐ Boot banner displays on reset
- ☐ Tick count increments every second in terminal output
- ☐ LD2 (PA5) appears to be ON (trace pin toggling too fast to see blink)

**If any check fails:** Debug before proceeding. Common issues:

| Symptom            | Likely Cause        | Fix                                      |
|--------------------|---------------------|------------------------------------------|
| No UART output     | Wrong COM port      | Check Device Manager                     |
| Garbled text       | Wrong baud rate     | Verify 115200 in terminal                |
| LD2 not on         | Timer not started   | Check <code>HAL_TIM_Base_Start_IT</code> |
| tick_count stays 0 | Callback not called | Verify interrupt enabled in NVIC         |

### Step 3.1: Oscilloscope Setup

#### Physical connections:

1. Connect oscilloscope probe to PA5 (TRACE pin / LD2)
2. Connect oscilloscope ground clip to GND header pin
3. Use short ground lead to minimize noise

#### Oscilloscope settings:

- Channel 1: DC coupling
- Voltage scale: 1 V/div
- Time scale: 500  $\mu$ s/div (to see ~1 ms period)
- Trigger: Rising edge, CH1, auto level

### Step 3.2: Capture and Measure Period

#### Measurement M1: Timer Period

1. Verify waveform is stable on screen
2. Use oscilloscope measurement function or cursors
3. Measure period between rising edges
4. Record minimum, maximum over at least 10 cycles

**Expected:** 1.000 ms  $\pm$  0.010 ms

#### If period is wrong:

- Check prescaler and period values in CubeMX configuration
- Verify timer clock source is correct (APB1 = 84 MHz)

### Step 3.3: Capture and Measure Pulse Width

#### Measurement M2: ISR Execution Time

1. Change time scale to 5  $\mu$ s/div or 10  $\mu$ s/div
2. Measure HIGH time (pulse width)
3. This represents the time spent inside the ISR

**Expected:** 2-10  $\mu\text{s}$  for simple ISR

**If pulse width is too long ( $> 50 \mu\text{s}$ ):**

- Check for printf calls inside ISR (remove them!)
- Check for blocking operations in callback

## Measurement Table (Required)

Complete this table during the lab:

| ID | Measurement        | Instrument   | Probe Point | Expected           | Measured            | Pass |
|----|--------------------|--------------|-------------|--------------------|---------------------|------|
| M1 | Timer period       | Oscilloscope | PA5 (TRACE) | 1.000 ms $\pm$ 1%  | _____ms             |      |
| M2 | ISR execution time | Oscilloscope | PA5 (TRACE) | $< 20 \mu\text{s}$ | _____ $\mu\text{s}$ |      |

## Completion Checklist

Before leaving the lab, verify you have accomplished:

- ☐ **Working demo:** UART log visible + trace pin toggling at 1 kHz
- ☐ **UART output:** Terminal showing boot banner and tick count
- ☐ **Measurement table:** Both measurements completed with values
- ☐ **Code understanding:** Can explain pointer, volatile, or struct usage in your code

## Cleanup Checklist

Before leaving:

- ☐ Power off board (disconnect USB)
- ☐ Disconnect oscilloscope probes
- ☐ Return shared equipment to designated location
- ☐ Push final code to repository
- ☐ Create git tag: `git tag lab01-demo && git push --tags`

## Common Issues and Solutions

| Issue                 | Symptom                      | Solution                                                               |
|-----------------------|------------------------------|------------------------------------------------------------------------|
| No serial output      | Terminal shows nothing       | Check COM port number, verify cable connected to ST-LINK USB           |
| Garbled characters    | Random symbols in terminal   | Match baud rate (115200) in both code and terminal                     |
| Tick count stuck at 0 | No increment                 | Enable TIM2 interrupt in NVIC, call <code>HAL_TIM_Base_Start_IT</code> |
| No trace waveform     | Oscilloscope shows flat line | Verify correct pin (PA5), check GPIO configured as output              |
| Very wide pulse       | Pulse > 100 $\mu$ s          | Remove printf from ISR, check for blocking calls                       |
| Period way off        | 10x or 100x expected         | Recalculate prescaler and period, check clock source                   |

## Key Takeaways

1. **UART logging makes debugging visible** - printf to serial terminal shows system state
2. **Trace pins reveal timing** - toggle GPIO in ISR to measure execution time on scope
3. **Volatile is mandatory for ISR variables** - compiler optimizations can break shared data
4. **Measure, don't assume** - oscilloscope shows what multimeter cannot
5. **This baseline project is your reference** - use it to verify hardware works in future labs